ETAS ISOLAR-A 12.7.0

Scripting Usecases

www.etas.com

Copyright

The data in this document may not be altered or amended without special notification from ETA S GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS ISOLAR-A 12.7.0 – Scripting Usecases R01 EN – 01.2025

Contents

1 About this Manual 4
1.1 Purpose 4
2 ISOLAR-A Scripting 5
2.1 Introduction 5
2.2 ISOLAR-A Scripting APIs 5
3 ISOLAR-A General Use Cases 6
3.1 Script Headers 6
3.2 Model Traversal and Reading 6
3.3 Model Creation – Creation of Elements 6
3.4 Searching for Elements 7
3.5 Report Generation 8
3.6 Other Use Cases 9
4 Contact Information 10

1 About this Manual

In this guide you will find information on how to use ISOLAR-A (Integrated SOLution for AutosaR – Application Software).

1.1 Purpose

The purpose of **this** document is to illustrate the use of ISOLAR-A Scripting through use cases with the help of
examples. This document helps developers to use ISOLAR-A APIs for their own use cases by demonstrating them.
Abstract: **AUTOSAR** is an evolving standard, which defines modular software architecture for automotive electronic
control units. It is designed to increase the ease of software re-use and the deployment on different hardware.
AUTOSAR increases the ease of software integration, but in a way mandates the use of tools to effectively
handle its complexity.

ISOLAR-A is the ETAS AUTOSAR Authoring Tool that assists users in designing application software to AUTOSAR
standards.

Who Should Read this Manual?
It is assumed that for developing AUTOSAR compatible systems and/or application software described by means
of AUTOSAR, you are interested to install and use ISOLAR-A.
You are recommended to read the ISOLAR-A Getting Started Guide first, before installing ISOLAR-A.

2 ISOLAR-A Scripting

ISOLAR-A Scripting environment supports both use case level and low level scripting functionality i.e. developers can directly access ISOLAR-A features via scripts and can also acces the underlying data model to process according to customized use cases.

2.1 Introduction

Scripting languages can be used to target specific runtime environments for rapid prototyping and automation.
Some of these languages provide high level of abstraction and are mostly interpreted, making them faster.

- ISOLAR-A Scripting Environment provides easy access to ISOLAR-A APIs
- Scripts can be used to extend the functionality of ISOLAR-A for variety of use cases
- Scripts can be managed centrally or at different location for the Scripting Environment
- APIs can be logically grouped and presented for easiness of the Script developers

Please note that a certain expertise with ARTOP Model, Java Script, Eclipse APIs is required to develop scripts for creating/ updating AUTOSAR configurations. Feature API calls are trivial and can be managed with basic JavaScript knowledge. Usage of JavaScript is recommended for development.This document focus on general use cases like – Creation, Traversal, Update, Search and Report Generation in AUTOSAR Models.

2.2 ISOLAR-A Scripting APIs

This document does not provide a detailed description of all the APIs themself, but provides example use cases and how to utilize the APIs of ISOLAR-A to achieve these use cases.The API documentation is available separately and is provided in ISOLAR- A API Documentation document provided with the release

3 ISOLAR-A General Use Cases

Here are some use case which are generally used with Scripting w.r.t. AUTOSAR.

3.1 Script Headers

Script headers are written to provide meta information about the scripts to the script framework. For example,
here is a script that is named “Get All Data Dictionary Elements” and it contributes a menu item with the described
image on all elements of type AtomicSwComponentType.

```
/*************************************************************************
******
* name : Get All Data Dictionary Elements
* image : platform:/plugin/org.eclipse.ease.ui/icons/eobj16/folder.png
* popup : enableFor(autosar40.swcomponent.components.AtomicSwComponentType)
* script-type : JavaScript
* script-engine : !org.eclipse.ease.javascript.nashorn
* description : Get All Data Dictionary Elements
*************************************************************************
*****/
```

On clicking the menu item -”Get All Data Dictionary Elements” on the AtomicSwComponentType, the script is run
with the selection as input.

3.2 Model Traversal and Reading

To access the Artop model in scripts, we can use the selection API. For example, to select a component and use it
in scripts for traversal, the following code snippet would work.

```
var selection = getSelection();
var component = selection.getFirstElement();
```

To use the getSelection() API, we have to use a load module command to load the UI related APIs. The load module
command is similar to an import statement in java or c.

```
loadModule('/System/Resources');
loadModule('/System/Platform');
loadModule('/System/UI');
loadModule('/Autosar Templates/AR Software/Component');
var selection = getSelection();
var component = selection.getFirstElement();
print("The selected Component Shortname is : " + component.getShortName() +"\n");
```

The selection will be passed on to the selection variable from the getSelection() API. The getFirstElement() API
returns the first model element in the selection list which can be used further on for other use cases.

3.3 Model Creation – Creation of Elements

Creation of new model elements in bulk is a common use case. For example, to create ports inside a component,
one can write a script function that creates the model elements from an Autosar Factory.
Model creation can be achieved using the AuotsarFactory Instance provided by the API

```
var port = getAutosar40FactoryInstance().createPPortPrototype();
port.setShortName(“PPort1”);
```

ETAS

ISOLAR-A v9.2 ISOLAR-A Scripting Use Cases 7
To use the **getAutosar40FactoryInstance()** API, a load module has to be called to load the Artop Utils. These
utils provide us with various options to work with the artop module.
loadModule('/Artop Utils/Artop');
Please note, this port object is not added to the actual model and persisted to file. To do this, we have to use
another API **addElements(EObject, EObject, String)** to attach the newly created object to the model.
addElement(EObject parentObject, EObject childObject, String feature);
Here parent object is the object to which the new child object has to be added. Feature is the structural feature to
which the child object is being added. In this example, feature would be "Ports" as we are adding a new port to a
component. So, the final API call will be -
addElement(component, port, "Ports");
After the execution of this line, the new port will be added to the specified component.

3.4 Searching for Elements

For operations to be performed on the model elements, sometimes we have to search for these elements in the
project. A project might contain a huge data model and the APIs have to traverse the model to find the component.
For example we want to find all the components in the project, we have to use the **getAllInstances()** API.
var compClass = getAutosar40FactoryInstance().createApplicationSwComponentType().getClass();
var components = getAllInstancesAsList(project, compClass ,true);
This API requires the classType of the object to be searched. So we can get the class type from the newly created
type from the factory using the getClass method.
getAllInstances() returns a list of all the components in the project. Further to check for a specific component,
this list can be iterated and checked for shortName.
1.5 Model Update
To update the model, we use specific model APIs. This depends on whether the use case is to add, delete, set or
remove from the data model. The APIs are –
1. 1. addElement(EObject, EObject, String);
2. 2. deleteFromModel(EObject);
3. 3. set(EObject, object, String);
4. 4. removeElement(EObject, EObject, String);
5.
Here EObject can be any element that can be passed to the API from the selection API explained in the first section.
String is the structural feature to which the element is being added or being set as reference. The structural
feature is also called as the "Role" to which the element is being added or set reference to. For example, ports are
added to the Component/Composition in the "Ports" role.
addElement() API has been demonstrated in the " Model Creation – Creation of Elements" on the previous page
deleteFromModel() API deletes the model element permanently. It would be removed from the file also.

set() and **removeElement()** API are used to set and remove references based on the "Role" described above.

3.5 Report Generation

Excel reports can be generated using ISOLAR-A scripting by traversing the model. For example we want to generate a report which contains the components with their ports and their associated data elements.

A general algorithm to perform this would be as follows –

1. Get the project
2. Create the report.xls file
3. Search all the components in project
4. Iterate through each component
5. Collect all ports and iterate
6. Write the data to file

Here is an example script that realises the above algorithm.

Lines 1-4 are general loadmodules which are needed for using some platform APIs.

Lines 5-6 take project as selection and use it as a variable.

```
/*****************************************************************************
******
* name : Generate Report
* image : platform:/plugin/org.eclipse.ease.ui/icons/eobj16/folder.png
* popup : enableFor(org.eclipse.core.resources.IProject)
* script-type : JavaScript
* script-engine : !org.eclipse.ease.javascript.nashorn
* description : Shows details of connectors
*****************************************************************************
*****/
1. loadModule('/ISOLAR-A Utils/ISOLAR Utils');
2. loadModule('/Artop Utils/Artop');
3. loadModule('/System/Resources');
4. loadModule('/System/UI');
5. var selection = getSelection();
6. var project = selection.getFirstElement();
7. print("start");
8.
9. var file = createFile("Report.xls");
//the createFile() API, this creates the .xls file. This API will work only if the loadMod-
ule(/System/Resources) is
//present.
10. var string = "\n\n\t\tComponent\tPort\tData Element\n";
11. var compClass = getAutosar40FactoryInstance().createApplicationSwCompo-
nentType().getClass();
12. var components = getAllInstancesAsList(project, compClass ,true);
13. //Searches the project for all Components.
14. var iterator = components.iterator();
15. while(iterator.hasNext()){
16. var component = iterator.next();
17. // main iterator for Components, within there is another iterator for ports at line 1
18. var ports = component.getPorts();
19. var portsIterator = ports.iterator();
20. while(portsIterator.hasNext())
21. var port = portsIterator.next();
//check for the type of Ports. (PPort or RPort) and according get the provider interface or
required interface. The
//data elements are then fetched from the getDataElements() API.
```

```
22. if(port.eClass().getName()=="RPortPrototype"){
var reqdInterf = port.getRequiredInterface();
var dataElements = reqdInterf.getDataElements();
var dataIterator = dataElements.iterator();
while(dataIterator.hasNext()){
writeInfo(file, component,port, dataIterator.next());
}
23. }else {
var provInterf = port.getProvidedInterface();
var dataElements = provInterf.getDataElements();
var dataIterator = dataElements.iterator();
while(dataIterator.hasNext()){
writeInfo(file, component,port, dataIterator.next());
}
24. }
25. }
26. }
27. print(string);
//writeLine method call which actually writes the string to the file.
28. writeLine(file, string, 2);
29. //function to build the string which has to be written to the file.
30. function writeInfo(file, component, port, dataElement){
31. string = string + "\t\t"+component.getShortName() + "\t" + port.getShortName() + "\t"
+ dataElement.
getShortName() + "\n";
32. }
```

3.6 Other Use Cases

The use cases described above are the most common use cases that are used in relation with AUTOSAR in ISOLAR. There can be other use cases that can also be accomplished with scripting. These may be file creation,copying, reading and writing from files etc. When working with a critical project, usage of scripts for these usecases should be done with caution as scripts can modify workspace content.

4 Contact Information

Technical Support

For details of your local sales office as well as your local technical support team and product hotlines, take a look at the website: www.etas.com/hotlines

ETAS Headquarters

ETAS GmbH
Borsigstraße 24
70469 Stuttgart
Germany

Phone: +49 711 3423-0
Fax: +49 711 3423-2106
Internet: www.etas.com

Figures

Tables

Index

E

ETAS, 6

Revision History